

Practical 1

```
In [2]: import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn import metrics
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import warnings
import seaborn as sns
import matplotlib.pyplot as plt
from math import *
warnings.filterwarnings("ignore")
```

Reading 'Uber' Dataset

```
In [3]: df = pd.read_csv("../data/uber.csv")
```

Reading head elements

```
In [4]: df.head()
```

```
Out[4]:
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dr
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	

Dropping Unnamed Columns

```
In [5]: df = df.drop(["Unnamed: 0", "key"], axis=1)
```

```
In [6]: df.head()
```

```
Out[6]:
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude
0	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.72321
1	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.75032
2	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.77264
3	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	40.80334
4	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	40.76124



Reading Data Information

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   fare_amount           200000 non-null float64
1   pickup_datetime       200000 non-null object
2   pickup_longitude      200000 non-null float64
3   pickup_latitude       200000 non-null float64
4   dropoff_longitude     199999 non-null float64
5   dropoff_latitude      199999 non-null float64
6   passenger_count       200000 non-null int64  
dtypes: float64(5), int64(1), object(1)
memory usage: 10.7+ MB
```

Transforming Data Time

```
In [8]: df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"], errors="coerce")
```

```
In [9]: df.describe().T
```

Out[9]:		count	mean	std	min	25%	50%	75%	
	fare_amount	200000.0	11.359955	9.901776	-52.000000	6.000000	8.500000	12.500000	
	pickup_longitude	200000.0	-72.527638	11.437787	-1340.648410	-73.992065	-73.981823	-73.967154	
	pickup_latitude	200000.0	39.935885	7.720539	-74.015515	40.734796	40.752592	40.767158	1
	dropoff_longitude	199999.0	-72.525292	13.117408	-3356.666300	-73.991407	-73.980093	-73.963658	1
	dropoff_latitude	199999.0	39.923890	6.794829	-881.985513	40.733823	40.753042	40.768001	
	passenger_count	200000.0	1.684535	1.385997	0.000000	1.000000	1.000000	2.000000	

Checking Null Values

```
In [10]: df.isna().sum()
```

```
Out[10]: fare_amount      0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude     1
dropoff_latitude      1
passenger_count      0
dtype: int64
```

Dropping Null Values

```
In [11]: df = df.dropna()
```

Transforming Coordinates to Distance using Haversine method

```
In [12]: def distance_transform(longitude1, latitude1, longitude2, latitude2):
distance = []
for pos in range(len(longitude1)):
    long1,lati1,long2,lati2 = map(radians,[longitude1[pos],latitude1[pos],longitude2[pos]
    dist_long = long2 - long1
    dist_lati = lati2 - lati1
    a = sin(dist_lati/2)**2 + cos(lati1) * cos(lati2) * sin(dist_long/2)**2
    c = 2 * asin(sqrt(a))*6371
    distance.append(c)
return distance
```

```
In [13]: df["distance_km"] = distance_transform(df["pickup_longitude"].to_numpy(), df["pickup_latitude"]
df["dropoff_longitude"].to_numpy(), df["dropoff_latitude"]
```

```
In [14]: df = df.assign(pickup_hr = df.pickup_datetime.dt.hour,
day= df.pickup_datetime.dt.day,
month = df.pickup_datetime.dt.month,
year = df.pickup_datetime.dt.year,
day_of_week = df.pickup_datetime.dt.dayofweek,
day_name=df.pickup_datetime.dt.day_name())
```

```
In [15]: df.head()
```

Out[15]:	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude
0	7.5	2015-05-07 19:52:06+00:00	-73.999817	40.738354	-73.999512	40.72321
1	7.7	2009-07-17 20:04:56+00:00	-73.994355	40.728225	-73.994710	40.75032
2	12.9	2009-08-24 21:45:00+00:00	-74.005043	40.740770	-73.962565	40.77264
3	5.3	2009-06-26 08:22:21+00:00	-73.976124	40.790844	-73.965316	40.80334
4	16.0	2014-08-28 17:47:00+00:00	-73.925023	40.744085	-73.973082	40.76124

Finding Outliers

```
In [16]: def find_outliers(df):
          q1 = df.quantile(0.25)
          q3 = df.quantile(0.75)
          IQR = q3-q1
          outliers = df[((df<(q1-1.5*IQR)) | (df>(q3+1.5*IQR)))]
          return outliers
```

```
In [17]: outliers = find_outliers(df['fare_amount'])
          print('number of outliers:' + str(len(outliers)))
          print('max outlier value:' + str(outliers.max()))
          print('min outlier value:' + str(outliers.min()))
          outliers
```

```
number of outliers:17166
max outlier value:499.0
min outlier value:-52.0
```

```
Out[17]: 6      24.50
          30      25.70
          34      39.50
          39      29.00
          48      56.80
          ...
          199976  49.70
          199977  43.50
          199982  57.33
          199985  24.00
          199997  30.90
          Name: fare_amount, Length: 17166, dtype: float64
```

```
In [18]: outliers = find_outliers(df['passenger_count'])
          print('number of outliers:' + str(len(outliers)))
          print('max outlier value:' + str(outliers.max()))
          print('min outlier value:' + str(outliers.min()))
          outliers
```

```
number of outliers:22557
max outlier value:208
min outlier value:4
```

```
Out[18]: 4      5
        6      5
        12     5
        24     5
        29     5
        ..
        199958  5
        199959  5
        199962  4
        199969  5
        199985  5
Name: passenger_count, Length: 22557, dtype: int64
```

Removing Outliers

```
In [19]: df.drop(df[df['distance_km'] == 0].index, inplace = True)
df.drop(df[df['distance_km'] > 60].index, inplace = True)
df.drop(df[df['fare_amount'] > 100].index, inplace = True)
df.drop(df[df['fare_amount'] < 0].index, inplace = True)
df.drop(df[df['passenger_count'] > 6].index, inplace = True)
```

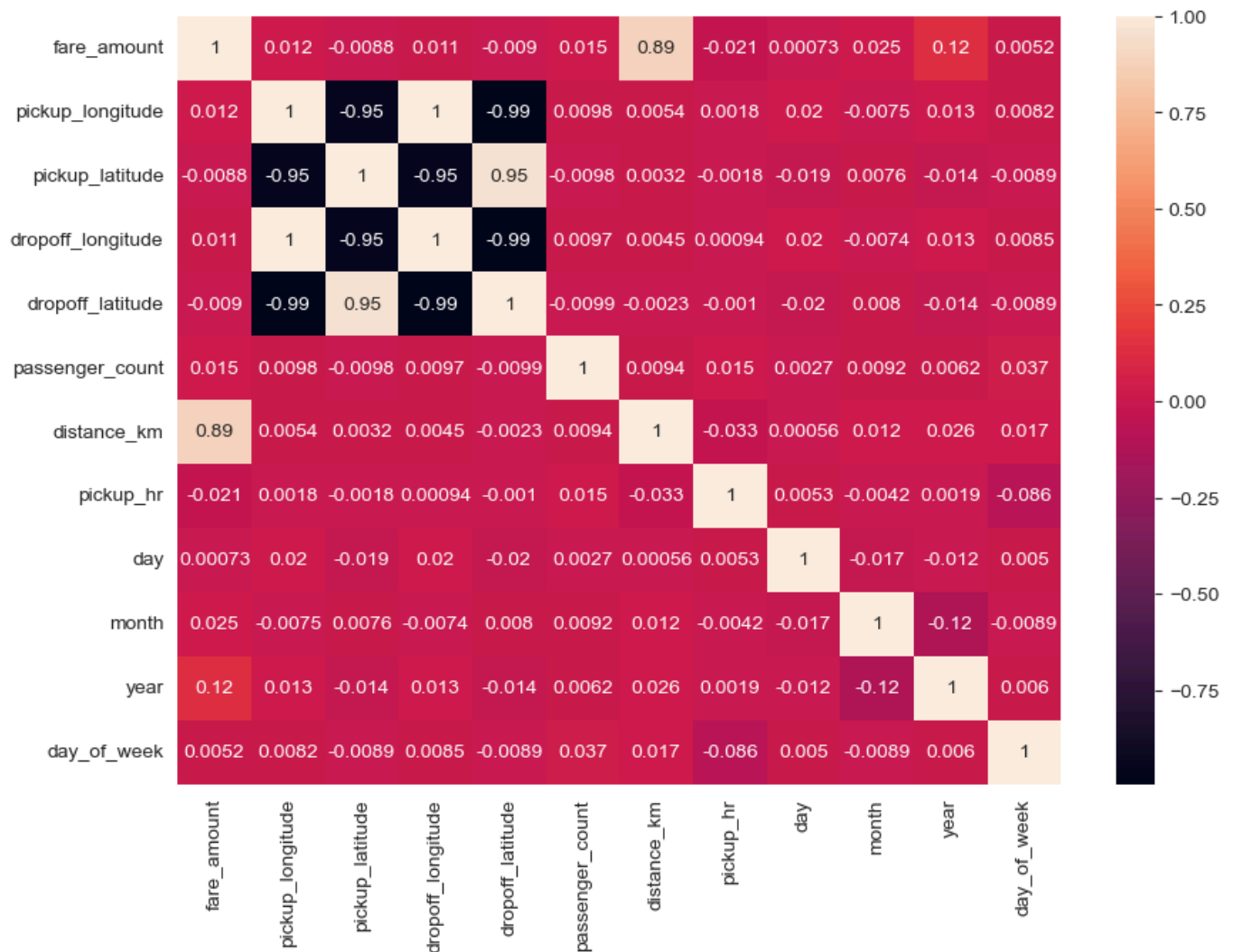
```
In [20]: df.head()
```

```
Out[20]:
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude
0	7.5	2015-05-07 19:52:06+00:00	-73.999817	40.738354	-73.999512	40.72321
1	7.7	2009-07-17 20:04:56+00:00	-73.994355	40.728225	-73.994710	40.75032
2	12.9	2009-08-24 21:45:00+00:00	-74.005043	40.740770	-73.962565	40.77264
3	5.3	2009-06-26 08:22:21+00:00	-73.976124	40.790844	-73.965316	40.80334
4	16.0	2014-08-28 17:47:00+00:00	-73.925023	40.744085	-73.973082	40.76124

Plotting HeatMap

```
In [21]: numeric_df = df.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()
plt.figure(figsize=(10, 7))
sns.heatmap(corr_matrix, annot=True)
plt.show()
```



Making Linear Regression Model

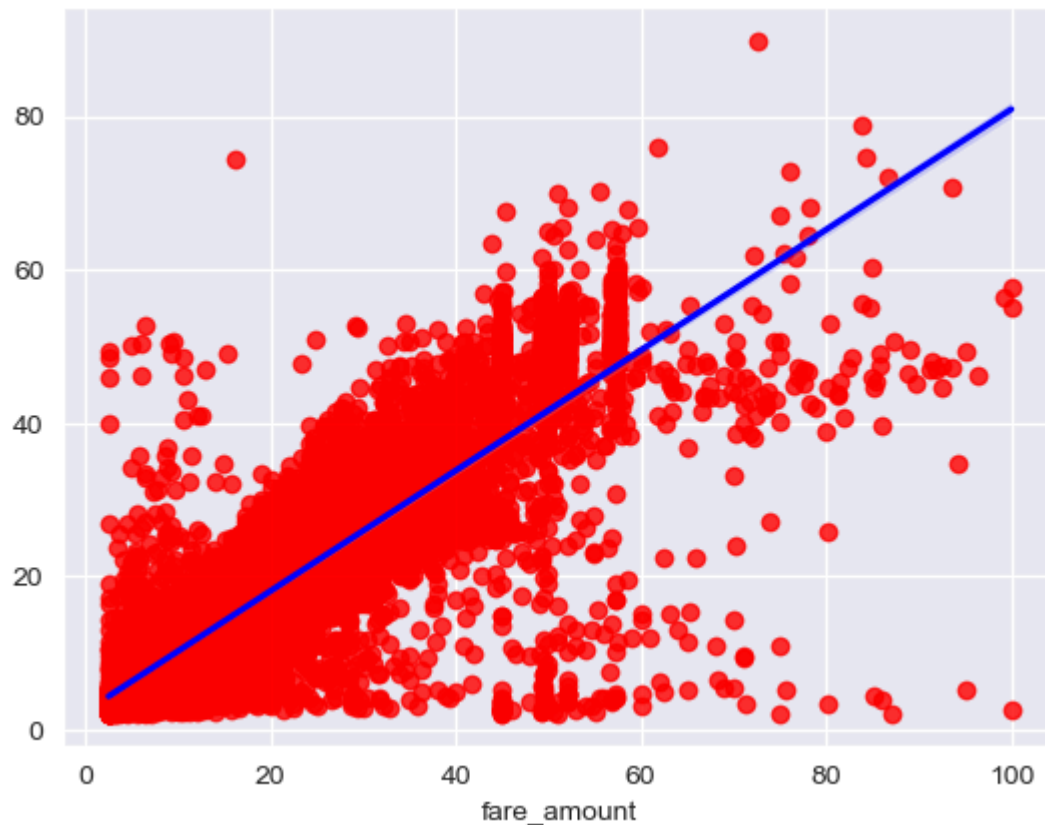
```
In [22]: x = df[["year", "distance_km"]]
y = df["fare_amount"]
scaler = StandardScaler()
x = scaler.fit_transform(x)
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
model = LinearRegression()
model.fit(x_train, y_train)
```

Out[22]: **LinearRegression** ⓘ ?

LinearRegression()

Plotting Graph

```
In [23]: y_pred = model.predict(x_test)
sns.regplot(x=y_test, y=y_pred, color="red", line_kws={"color" : "blue"})
plt.show()
```



```
In [24]: print(f"Mean absolute error {metrics.mean_absolute_error(y_test, y_pred)}")
print(f"Mean squared error {metrics.mean_squared_error(y_test, y_pred)}")
print(f"Root mean squared error {np.sqrt(metrics.mean_squared_error(y_test, y_pred))}")
```

Mean absolute error 2.2438795034594903

Mean squared error 18.323819106454565

Root mean squared error 4.280633026370581

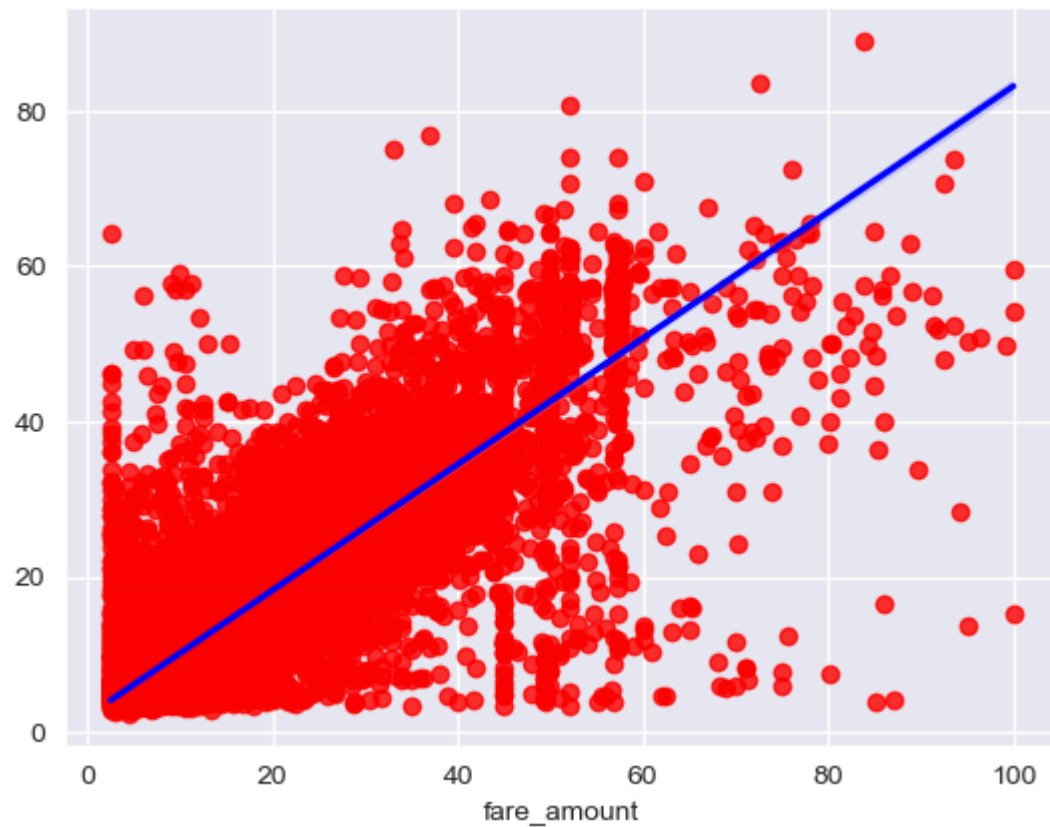
Making Random Forest Model

```
In [25]: model = RandomForestRegressor()
model.fit(x_train, y_train)
```

```
Out[25]: ▼ RandomForestRegressor ⓘ ?
RandomForestRegressor()
```

Plotting Graph

```
In [26]: y_pred = model.predict(x_test)
sns.regplot(x=y_test, y=y_pred, color="red", line_kws={"color" : "blue"})
plt.show()
```



```
In [27]: print(f"Mean absolute error {metrics.mean_absolute_error(y_test, y_pred)}")
print(f"Mean squared error {metrics.mean_squared_error(y_test, y_pred)}")
print(f"Root mean squared error {np.sqrt(metrics.mean_squared_error(y_test, y_pred))}")
```

Mean absolute error 2.5002183202816988

Mean squared error 21.301173090605587

Root mean squared error 4.615319392047054

In []: